LAB 07

Fuzzing for Input Validation Bugs

Pet Boarding Application

Experiment: Property-based fuzz testing using Hypothesis
Programming Language: Python | Testing Tools: Hypothesis and Pytest

Software Engineering Laboratory — Practical Report

1. Objective

The objective of this lab is to explore fuzz testing as a security and quality assurance technique by identifying and fixing input-validation bugs in a Python application using the Hypothesis fuzzing library. The experiment checks how the Pet Boarding application handles random, unexpected, malformed, and invalid inputs, and improves its robustness through defensive programming.

2. Software Requirements and Tools

Python 3.13.5 was used as the programming language, Hypothesis 6.170.15 as the property-based fuzzing library, and Pytest 8.3.4 as the test framework. The work was carried out in a Windows PowerShell and VS Code environment.

3. Introduction to Fuzz Testing

Fuzz testing is a dynamic testing technique in which a program is supplied with random, unexpected, or malformed data to discover crashes and other incorrect behavior. Manual tests usually cover expected inputs, while fuzz testing explores edge cases that may not have been anticipated. Hypothesis supports property-based testing: the developer describes a property, such as a function not raising an exception, and Hypothesis automatically generates inputs to try to violate that property. It can also shrink a failing example to a simpler input that makes the defect easier to diagnose.

4. Application Functions

Function	Purpose
parse_booking(text)	Splits a pet-and-days input using ':', trims and title-cases the pet name, and converts days to a numbe
stay_average(days_csv)	Splits comma-separated stay durations, converts them to integers, and calculates the integer average
payable_fee(text)	Splits fee and discount using '|', applies 18% to the fee, subtracts the discount, and prevents a negativ

5. Hypothesis Test Implementation

The test file imports the three application functions and uses the Hypothesis @given decorator with st.one_of(st.text(), st.none()). This strategy supplies generated text values as well as None. Each test invokes its target function and reports a failure if an exception is raised. The tests are named test_parse_booking_no_crash, test_stay_average_no_crash, and test_payable_fee_no_crash. Pytest discovers and executes these tests.

6. Bugs Identified Before the Fix

The initial fuzzing run failed all three tests. Hypothesis exposed malformed inputs that the original functions did not validate. For parse_booking, examples included ':' and an empty string, causing unpacking or float-conversion errors; None caused an attribute error. For stay_average, an empty string caused integer conversion to fail and None caused an attribute error. For payable_fee, inputs such as '|' and an empty string caused unpacking or numeric conversion errors, while None caused an attribute error. These results showed that the original functions assumed inputs were correctly formatted.

7. Fixes and Defensive Input Validation

The application was updated to validate input types and delimiters before processing. The functions check that the supplied value is a string, verify the expected delimiter count, reject missing required fields, and catch ValueError or TypeError during numeric conversion. The booking function rejects empty pet names and invalid

or negative day values. The stay-average function rejects empty input, invalid integer entries, and negative stay lengths. The fee function validates both numeric values and rejects negative or non-finite fee and discount values. Invalid inputs return None instead of allowing an exception to terminate execution. Valid input behavior is retained.

8. Test Results

Stage	parse_booking	stay_average	payable_fee	Overall
Before fix	FAILED	FAILED	FAILED	3 failed
After fix	PASSED	PASSED	PASSED	3 passed

After applying the fixes, the test command completed successfully with 3 passed in 0.99s. The fixed application was also executed with valid sample inputs: dog:5, 2,5,7, and 1500|100. The observed results were ('Dog', 5.0), an average stay of 4, and a payable fee of 1670.0.

9. Experiment Screenshots — Figure 1

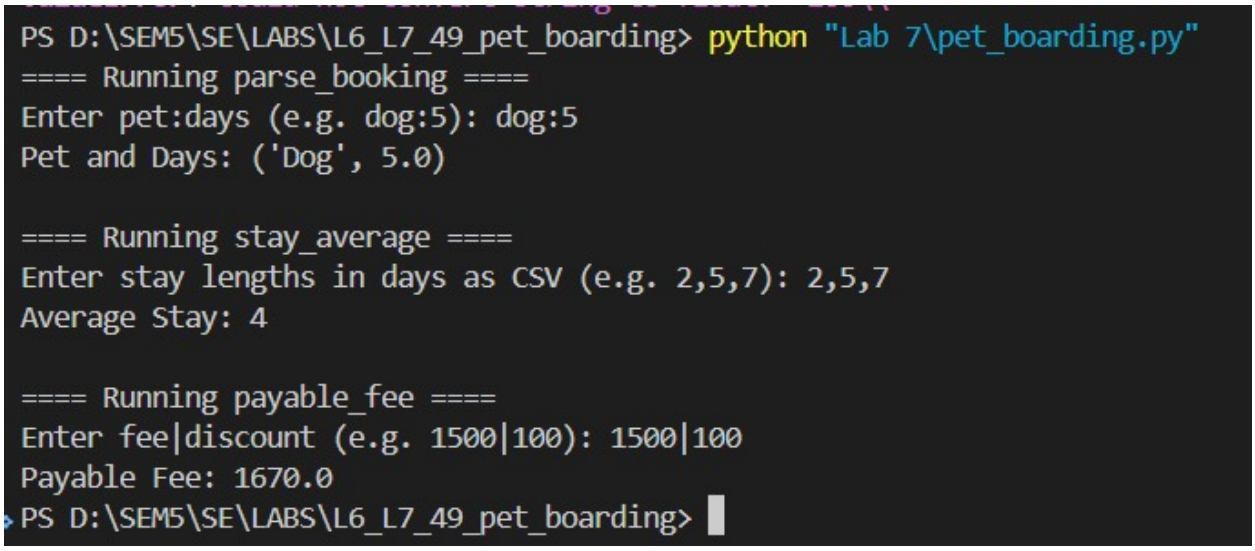


Figure 1. Original Pet Boarding application execution before fuzz testing. The application processes valid sample inputs and displays the expected results.

9. Experiment Screenshots — Figure 2

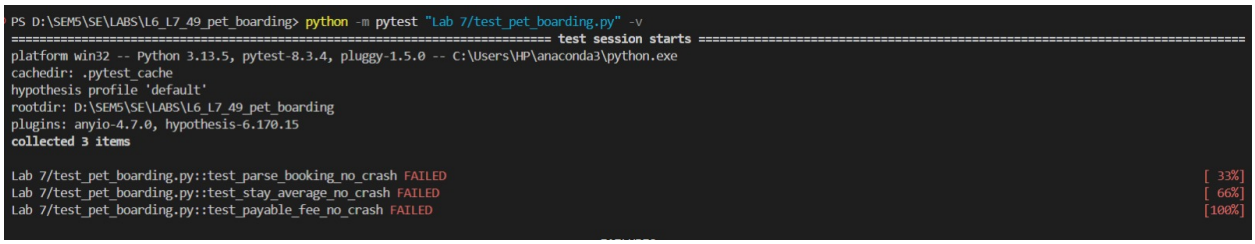


Figure 2. Hypothesis fuzz testing failures before the fixes. All three tests fail, demonstrating that unexpected inputs expose input-validation defects.

9. Experiment Screenshots — Figure 3

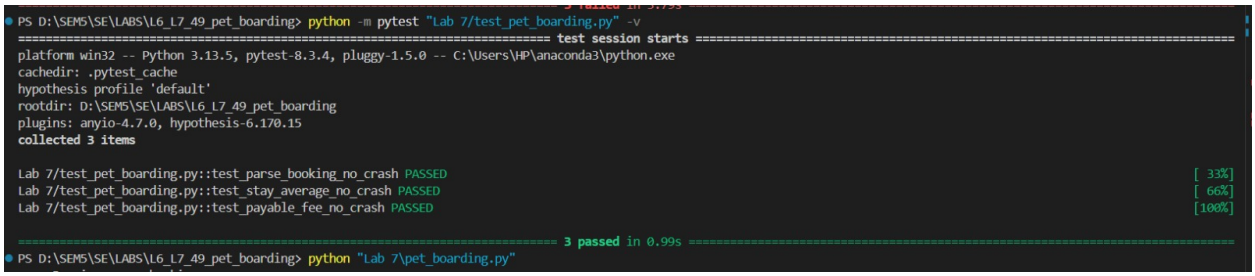


Figure 3. Hypothesis fuzz testing after the fixes. All three tests pass; the test run reports 3 passed in 0.99s.

9. Experiment Screenshots — Figure 4

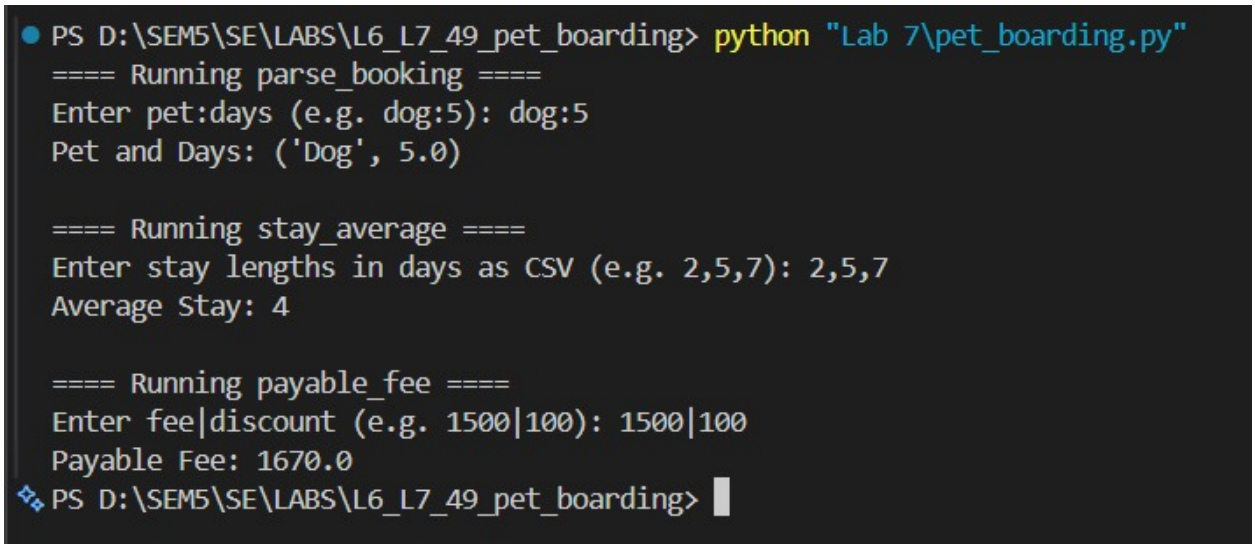


Figure 4. Fixed Pet Boarding application execution. The corrected application continues to process valid sample inputs correctly.

10. Conclusion

This experiment demonstrated how property-based fuzz testing can uncover input-validation bugs that may not appear during ordinary execution. Hypothesis generated unexpected values for all three Pet Boarding functions and revealed failures involving malformed delimiters, empty strings, invalid numeric values, and None inputs. Defensive validation and exception handling were added to the application. After the corrections, all three fuzz tests passed, and the application continued to produce the expected results for valid sample inputs. The lab illustrates the value of fuzz testing for improving software robustness and reliability.

11. Deliverables

The completed submission includes this PDF report with experiment screenshots, the fixed pet_boarding.py application script, and the updated test_pet_boarding.py Hypothesis test script.